Raspberry Pi as the MQTT gateway

NodeMCU publisher → Pi broker → laptop subscriber, plus packet capture on the Pi

This is step 1 and 2 of the TinyIDS experiment. In TinyIDS the Raspberry Pi is the **gateway**: all sensor traffic must flow *into* the Pi, because that is where the **IDS (Intrusion Detection System)** will watch the packets. So the **MQTT (Message Queuing Telemetry Transport)** broker runs on the Pi.

The setup

`NodeMCU (publisher)`
↓ Wi-Fi, MQTT, topic `test/01`
`Raspberry Pi (Mosquitto broker = gateway)`
↓ MQTT
`Windows laptop (sub.py)`

Device	Role	IP (college Wi-Fi, 06 Nov 2026)
NodeMCU (ESP8266)	publisher, topic `test/01`	172.29.1.103
Raspberry Pi 4	broker (Mosquitto) = gateway	172.29.1.135
Windows laptop	subscriber (`sub.py`)	172.29.1.102

All three are on the college Wi-Fi (IIITA). The IPs are given out automatically (DHCP = Dynamic Host Configuration Protocol) and **change from day to day**; on a phone hotspot they usually stay the same. Find the Pi's current IP with `hostname -I` on the Pi.

Files used (from the `mqtt_test` folder; no new code was needed):

- `nodemcu_mqtt_pub/nodemcu_mqtt_pub.ino` on the NodeMCU
- `sub.py` on the laptop

Step 1: Check the Pi's broker

The lab Pi already has Mosquitto installed and configured. Connect to the Pi (monitor + keyboard, or SSH = Secure Shell: `ssh pi@<pi-ip>`) and check:

```
sudo systemctl status mosquitto            # should say: active (running)
cat /etc/mosquitto/conf.d/local.conf
```

`local.conf` contains:

```
listener 1883
allow_anonymous true
```

- `listener 1883` = accept connections from *other* devices on port 1883 (not only from the Pi itself).
- `allow_anonymous true` = no username/password needed (fine for a lab test).

Don't add a second config file with another `listener 1883`: two listeners on the same port stop Mosquitto from starting.

Fresh Pi without Mosquitto? `sudo apt install -y mosquitto mosquitto-clients`, then create `local.conf` with the two lines above and `sudo systemctl restart mosquitto`. (Needs the proxy on the college network: see Step 5.)

Step 2: Point the NodeMCU at the Pi

In `nodemcu_mqtt_pub.ino`, keep exactly one `BROKER` line uncommented and set it to the Pi's IP:

```
//const char* BROKER = "172.29.0.43";       // laptop broker (not used now)
const char* BROKER = "172.29.1.135";        // the Pi
// const char* BROKER = "broker.hivemq.com"; // public broker (hotspot only)
```

Upload, then check the Serial Monitor (9600 baud): `Connecting to MQTT broker 172.29.1.135 ... OK` followed by `sent {...}` every 2 seconds.

Step 3: Receive on the Pi

On the Pi:

```
mosquitto_sub -h localhost -t test/01 -v
```

`-h localhost` = the broker on this Pi, `-t test/01` = the topic, `-v` = also print the topic name. Output every 2 seconds:

```
test/01 {"uptime_s":733,"rssi_dbm":-76,"free_heap":50112,"a0":5}
test/01 {"uptime_s":735,"rssi_dbm":-76,"free_heap":50112,"a0":4}
```

Stop with Ctrl+C.

Step 4: Receive on the Windows laptop

`sub.py` takes the broker's address as an argument (no need to edit the file):

```
cd C:\Users\HOME\Documents\iot\project_final\mqtt_test
python sub.py 172.29.1.135
```

```
Connected to 172.29.1.135, listening on 'test/01' ...
18:58:40  test/01  {"uptime_s":127,"rssi_dbm":-75,"free_heap":50112,"a0":4}
18:58:42  test/01  {"uptime_s":129,"rssi_dbm":-76,"free_heap":50112,"a0":5}
```

`python sub.py` with no argument uses `localhost` (a broker on the laptop itself).

Result: the same message reaches both the Pi and the laptop. The broker on the Pi forwards every message to everyone subscribed to `test/01`.

Step 5: Internet on the Pi through the college proxy (for installing software)

On the college network, the Pi can only reach the internet through the proxy (host `172.31.2.3`, port `8080`, no username/password). Without it, `apt` fails with `403 Forbidden`.

Turn the proxy ON for apt (the Debian/Raspberry Pi package installer):

```
echo 'Acquire::http::Proxy "http://172.31.2.3:8080";
Acquire::https::Proxy "http://172.31.2.3:8080";' \
  | sudo tee /etc/apt/apt.conf.d/95proxy
```

This creates one small file. Test: `sudo apt update` or `sudo apt install tcpdump`.

Turn it OFF / delete it (e.g. when the Pi is on a phone hotspot, where the proxy does not exist):

```
sudo rm /etc/apt/apt.conf.d/95proxy
```

Check whether it is on: `cat /etc/apt/apt.conf.d/95proxy` ("No such file" = off).

For a single command that needs the internet (e.g. `curl`), you can pass the proxy just for that command with `-x http://172.31.2.3:8080`. Nothing is saved, so there is nothing to undo.

Step 6: Fix the Pi's clock

Problem: the Pi thought it was still the previous day. `apt` complained `Verifying signature: Not live until 2026-11-06T10:17:25Z`. The Pi has no battery-backed clock and normally sets its time from the internet using **NTP (Network Time Protocol)**; the college network blocks NTP, so the clock goes wrong. Wrong time = wrong timestamps in logs and alerts, which matter for the report.

Fix: read the current time from Google's web server (through the proxy) and set the clock to it:

```
sudo date -s "$(curl -sI -x http://172.31.2.3:8080 https://www.google.com \
  | grep -i '^date:' | cut -d' ' -f2-)"
date
```

How it works: `curl -sI` asks Google only for the response *headers*; one header is `date: Fri, 06 Nov 2026 13:38:55 GMT`; `grep`/`cut` keep just the date part; `date -s` sets the Pi's clock to it. Result:

```
Friday 06 November 2026 07:08:55 PM IST
```

How to revert / undo:

- Nothing permanent was changed: the clock goes back to automatic behaviour on the next reboot.
- To hand control back to automatic time sync right away: `sudo timedatectl set-ntp true`
- Check the time-sync status: `timedatectl` (look at "System clock synchronized").
- On a phone hotspot NTP works, so the clock fixes itself; on college Wi-Fi run the fix again after each reboot.

Step 7: Capture the traffic on the Pi (base of the IDS)

The IDS will work by capturing packets on the Pi. **tcpdump** is a command-line packet capture tool (like Wireshark without the window):

```
sudo tcpdump -i wlan0 -n port 1883 -c 20
```

`-i wlan0` = listen on the Pi's Wi-Fi; `-n` = show IPs, not names; `port 1883` = only MQTT traffic; `-c 20` = stop after 20 packets.

Output (part):

```
19:09:18.630465 IP 172.29.1.103.62463 > 172.29.1.135.1883: Flags [P.], ... length 67
19:09:18.630490 IP 172.29.1.135.1883 > 172.29.1.103.62463: Flags [.], ack 67, ... length 0
19:09:18.630593 IP 172.29.1.135.1883 > 172.29.1.102.57558: Flags [P.], ... length 67
19:09:18.936755 IP 172.29.1.102.57558 > 172.29.1.135.1883: Flags [.], ack 67, ... length 0
...
19:09:25.170392 IP 172.29.1.103.62463 > 172.29.1.135.1883: Flags [P.], ... length 2
19:09:25.170660 IP 172.29.1.135.1883 > 172.29.1.103.62463: Flags [P.], ... length 2
20 packets captured
```

How to read it (every 2 seconds the same pattern of 4 packets):

Line	Meaning
`.103 > .135 [P.] len 67`	NodeMCU sends one MQTT **PUBLISH** (67 bytes = the JSON message) to the Pi.
`.135 > .103 [.] len 0`	Pi's TCP **ACK**: "got it" (no data).
`.135 > .102 [P.] len 67`	Pi **forwards** the same 67-byte message to the laptop (subscriber).
`.102 > .135 [.] len 0`	Laptop's ACK.
`len 2` packets	MQTT **PINGREQ / PINGRESP**: a tiny "are you still there?" keep-alive between NodeMCU and Pi.

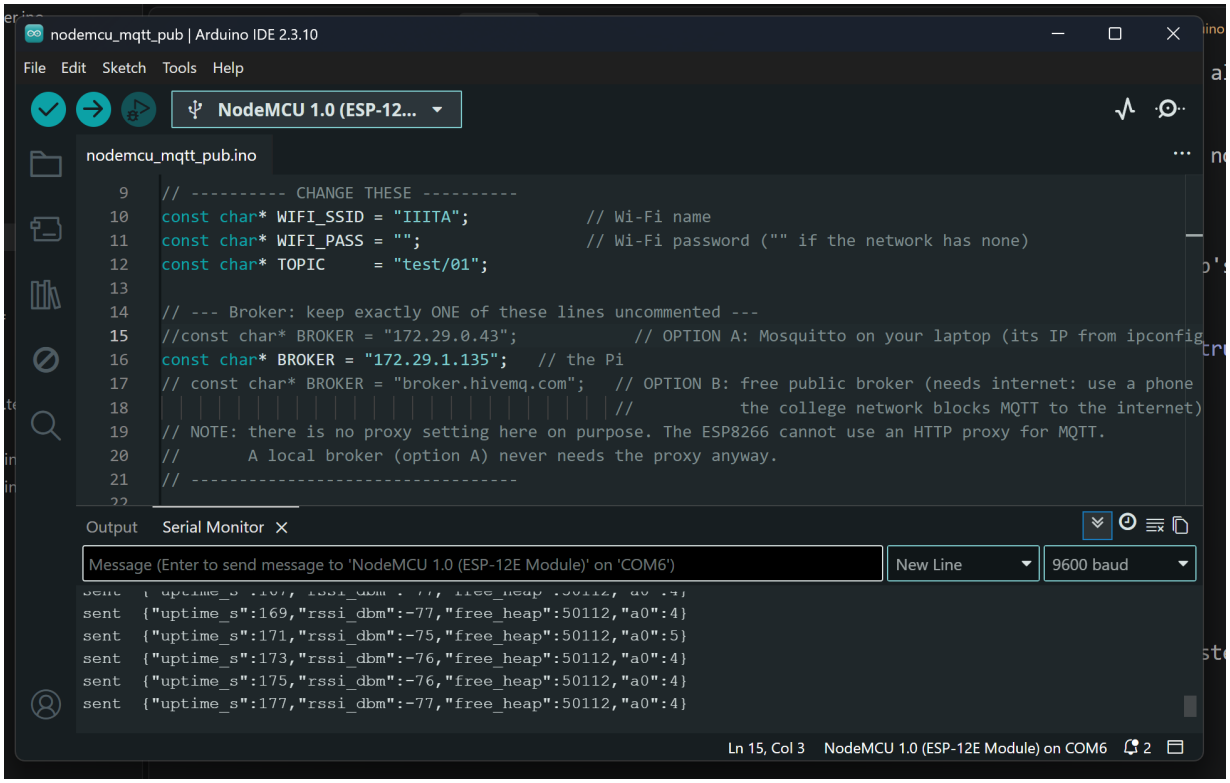
- `Flags [P.]` = **PSH + ACK** (TCP flags "push data now" + "acknowledge"); `Flags [.]` = ACK only. These are exactly the flag columns of the CICIoT2023 dataset (`psh_flag_number`, `ack_flag_number`). This is the bridge between the dataset and the live traffic: the live IDS will compute the same kind of features from these packets.
- `62463`, `57558` = random "source ports" the NodeMCU and laptop picked for their connection; `1883` = MQTT port.
- `0 packets dropped by kernel` = the Pi kept up with the traffic.

Troubleshooting

Symptom	Fix
NodeMCU: `failed (state -2)`	Wrong Pi IP (check `hostname -I` on the Pi), or Mosquitto not running (`sudo systemctl restart mosquitto`).
`sub.py`: connection refused / timeout	Same as above; also check the laptop is on the same Wi-Fi.
`apt`: `403 Forbidden`	Proxy not set for apt (Step 5).
`apt`: `Not live until ...`	Pi clock is wrong (Step 6).
`tcpdump`: command not found	`sudo apt install tcpdump` (with the proxy on).

Result (tested, 06 November 2026)

NodeMCU sketch with `BROKER = "172.29.1.135"` (the Pi) and its Serial Monitor showing each message sent:



At the same time the messages arrived on the Pi (`mosquitto_sub`, Step 3), on the laptop (`python sub.py 172.29.1.135`, Step 4), and the packets were visible in `tcpdump` on the Pi (Step 7).